ExoHabitAI

Exoplanet Habitability Predictor

Milestone 3 — Module 5 & Module 6 Documentation

Project	ExoHabitAI — AI-Powered Exoplanet Habitability Predictor
Organisation	Infosys Internship Programme 2026
Dataset	NASA Exoplanet Archive (39,315 confirmed planets)
Best Model	Random Forest Tuned — AUC 0.906, F1 Score 0.71
Backend	Python, Flask 3.1, scikit-learn 1.8, joblib
Frontend	React.js, Framer Motion, Three.js, Canvas API
Modules	Module 5: Backend API | Module 6: Frontend UI

Table of Contents

1. Module 5: Backend API Integration
1.1 Objective
1.2 Folder Structure
1.3 Model Loading
1.4 API Endpoints
1.5 Input Validation and Feature Engineering
1.6 Response Structure and Error Handling
1.7 CORS and Testing
2. Module 6: Frontend UI Development
2.1 Objective
2.2 Technology Stack
2.3 Folder Structure and Components
2.4 Application Flow
2.5 Authentication Page
2.6 Prediction Console
2.7 Result Panel and 3D Visualization
2.8 History and About Sections
3. ML Model Performance Summary
4. End-to-End Integration Flow

Module 5: Backend API Integration (Flask)

1.1 Objective

The objective of Module 5 is to expose the trained Random Forest machine learning model through a Flask REST API. The API accepts raw exoplanet parameters as JSON, applies the same preprocessing pipeline used during training including feature engineering and StandardScaler transformation, and returns a structured habitability prediction with a confidence score.

1.2 Folder Structure

The backend follows a clean separation of concerns with the Flask application, utility helpers, and the trained model stored in separate files.

File / Folder	Purpose
backend/App.py	Main Flask application — routes and API logic
backend/utils.py	Input validation and feature engineering
backend/requirements.txt	Python dependency list
backend/model.pkl	Copy of trained final_model.pkl (47 MB)
models/final_model.pkl	Trained RF Tuned model
models/scaler.pkl	Fitted StandardScaler (16 features)
models/feature_cols.pkl	Ordered list of 16 feature column names
data/processed/habitability_ranked.csv	Pre-ranked habitability scores for /rank

1.3 Model Loading

The Flask application loads the trained model, scaler, and feature column list at startup using joblib. The model path checks for backend/model.pkl first, then falls back to models/final_model.pkl.

```
MODEL_PATH = backend/model.pkl (primary) → models/final_model.pkl (fallback) SCALER
= models/scaler.pkl FEAT_COLS = models/feature_cols.pkl (16 features)
```

1.4 API Endpoints

The API exposes four endpoints. All responses return structured JSON with a status field set to either success or error.

Endpoint	Method	Purpose
GET /	GET	Health check — returns API info and version
GET /health	GET	Returns model_loaded status as true or false
POST /predict	POST	Predicts habitability for a single exoplanet
GET /rank	GET	Returns top-N ranked habitable planets

POST /predict — Request Format

```
{ "pl_orbper" : 365.25, // Orbital period in days "pl_orbsmax" : 1.0, // Semi-major axis in AU "st_teff" : 5778, // Star temperature in Kelvin "st_met" : 0.0, // Star metallicity in dex "st_spectype" : "G" // Spectral type }
```

POST /predict — Response Format

```
{ "status" : "success", "prediction" : 1, "label" : "Potentially Habitable", "habitability_probability" : 0.8712, "confidence" : "87.12%" }
```

GET /rank — Query Parameters

- n=10 — Number of planets to return (1 to 1000)
- only_habitable=true — Filter to habitable planets only
- Example: GET /rank?n=5&only;_habitable=true

1.5 Input Validation and Feature Engineering

The utils.py module validates the incoming JSON and builds the scaled feature vector. All five required fields are checked for presence, type, and range before prediction.

Field	Type	Min	Max	Description
pl_orbper	float	0.01	1e9	Orbital period (days)
pl_orbsmax	float	0.001	100.0	Semi-major axis (AU)
st_teff	float	300.0	60000	Star temperature (K)
st_met	float	-5.0	10.0	Star metallicity (dex)
st_spectype	str	—	—	G / K / M / F / B / D / L / T / W / m

After validation, build_feature_vector constructs the 16-feature input:

- Computes stellar_compatibility using star temperature and luminosity proxy
- Computes orbital_stability using orbital period and semi-major axis
- One-hot encodes spectral type into 10 binary columns
- Aligns columns to the saved feature_cols.pkl order
- Applies StandardScaler transform loaded from scaler.pkl
- Returns scaled 1 x 16 DataFrame ready for model.predict()

1.6 Response Structure and Error Handling

All endpoints return consistent JSON with a status field. HTTP status codes reflect the outcome.

- 200 OK — Prediction successful, returns label, probability, and confidence
- 400 Bad Request — Missing field, invalid type, or value out of range
- 500 Internal Server Error — Model not loaded or prediction exception

1.7 CORS Configuration and Testing

Cross-Origin Resource Sharing is enabled for the React development server on localhost:3000. This prevents browser security errors when the frontend calls the Flask API.

```
CORS(app, origins=['http://localhost:3000', 'http://127.0.0.1:3000'])
```

- Browser — GET / and GET /health confirmed model load status
- PowerShell Invoke-WebRequest — POST /predict tested with Earth-like values
- React Frontend — full end-to-end flow via Fetch API

Module 6: Frontend UI Development (React.js)

2.1 Objective

Module 6 builds a space-themed web interface for ExoHabitAI using React.js. The frontend provides a cinematic story intro, user authentication, a multi-parameter prediction console, real-time 3D planet visualization, and prediction history — all connected to the Flask backend via the Fetch API.

2.2 Technology Stack

The following technologies were used to build the frontend. Each was chosen for a specific role in the application.

Technology	Version	Role in ExoHabitAI
React.js	19.2.4	Component architecture and hooks (useState, useEffect, useCallback, useRef)
Framer Motion	12.38.0	Page transitions, gauge animations, hover and tap effects on buttons
Three.js	0.183.2	3D planet rendering via WebGL in the result panel
@react-three/fiber	9.5.0	React renderer for Three.js — declarative 3D scene setup
@react-three/drei	10.7.7	OrbitControls and Sphere helpers used in PlanetVisualization.jsx
Canvas API	Native	Custom starfield renderer in GalaxyBackground.jsx and StoryPage.jsx
CSS (Vanilla)	Native	Full design system — glassmorphism panels, keyframe animations
Fetch API	Native	POST /predict to Flask backend — async, no page reload
localStorage	Native	Prediction history (exohistory) and session (exo_session) persistence

2.3 Folder Structure and Components

Component	Purpose
App.js	Root — manages flow state (story to auth to dashboard)
GalaxyBackground.jsx	Canvas animated starfield with 300 stars and shooting stars
StoryPage.jsx	Cinematic 24-second intro with timed text reveals
AuthPage.jsx	Login and Register with localStorage session management
Navbar.jsx	Fixed navigation bar with mobile hamburger menu
HeroSection.jsx	Landing page with animated counter and CTA buttons
GalaxyNavigation.jsx	Sidebar planet-themed navigation panel

Component	Purpose
PredictionPanel.jsx	10-field parameter form with validation and API call
ResultPanel.jsx	Prediction output — verdict, 3D planet, gauge, analysis
HabitabilityMeter.jsx	Animated SVG circular gauge showing probability
PlanetVisualization.jsx	3D planet with Three.js — color changes by habitability
HistoryPanel.jsx	Prediction log stored and read from localStorage
AboutSection.jsx	Project info cards and technology stack
LoadingOverlay.jsx	Full-screen loading animation during API call

2.4 Application Flow

The app uses a three-stage flow managed via React useState and localStorage. AnimatePresence with mode='wait' ensures clean transitions between stages.

Stage	Trigger Condition	What Happens
story	First visit — no session, no story_seen	24-second cinematic text animation
auth	Story seen but not logged in	Login / Register / Guest access
dashboard	Valid session exists in localStorage	Full prediction dashboard loads

2.5 Authentication Page

The auth page provides Sign In, Register, and Guest access. Credentials are stored in localStorage. No server-side authentication is used. A Continue as Guest button allows immediate access without registration.

- Sign In — validates email and password against localStorage exo_user
- Register — saves name, email, and password to localStorage
- Guest — auto-login as Guest Commander with no credentials required
- Session stored in localStorage exo_session as JSON object
- Auto-login on refresh if session already exists

2.6 Prediction Console

The prediction console is the core interface styled as a mission control terminal. It accepts 10 planetary and stellar parameters, validates required fields client-side, then sends 5 required fields to POST /predict.

Field	Label	Unit	Required
pl_rade	Planet Radius	R Earth	No
pl_bmasse	Planet Mass	M Earth	No

Field	Label	Unit	Required
pl_orbper	Orbital Period	days	Yes
pl_orbsmax	Semi-Major Axis	AU	Yes
pl_eqt	Surface Temp	K	No
st_teff	Star Temperature	K	Yes
st_met	Star Metallicity	dex	Yes
st_lum	Luminosity	log L	No
st_rad	Star Radius	R Sun	No
st_spectype	Spectral Type	G/K/M	Yes

All number inputs use type='text' with inputMode='decimal' to allow free decimal entry such as 365.25 or -0.1 without browser step restrictions.

2.7 Result Panel and 3D Visualization

After prediction, the ResultPanel displays a complete habitability report.

- Verdict Badge — HABITABLE or NON-HABITABLE with confidence percentage
- 3D Planet (Three.js) — planet color changes based on habitability probability
- Circular Gauge — animated SVG arc showing probability from 0 to 100 percent
- Factor Analysis — star temp, orbit, radius, spectral type marked as pass or fail
- Scientific Analysis — natural language explanation of the prediction result
- Input Summary — all entered values displayed in a key-value list

The 3D planet uses @react-three/fiber with OrbitControls for auto-rotation. An atmosphere glow, ring, and orbiting moon are included for visual richness.

2.8 History and About Sections

Every successful prediction is automatically saved to localStorage under exohistory with a maximum of 20 entries. The History panel shows all predictions in a table with mission ID, parameters, probability, and verdict. Entries persist across sessions.

The About section contains six cards covering the ML model, science criteria, backend, performance metrics, data source, and frontend technology.

ML Model Performance Summary

3.1 Dataset and Training Setup

- Dataset: NASA Exoplanet Archive — 39,315 confirmed exoplanets
- Features used: 16 (pl_rade dropped to prevent data leakage)
- Target: habitability — 1 if pl_rade < 2.0 AND st_teff between 4000 and 7000 K
- Class balance: 24.53% habitable (9,644), 75.47% non-habitable (29,671)
- Train/Test split: 80% training, 20% testing, stratified by class
- Cross-validation: 5-fold on training set

3.2 Model Comparison

Model	Accuracy	Precision	Recall	F1	AUC
Logistic Regression	0.6274	0.3616	0.6781	0.4717	0.7105
Decision Tree	0.7459	0.4856	0.6013	0.5373	0.7607
Random Forest	0.7491	0.4926	0.7579	0.5971	0.8297
XGBoost	0.7539	0.4990	0.7880	0.6111	0.8420
RF Tuned (Winner)	0.8576	0.7086	0.7123	0.7104	0.9060
XGBoost Tuned	0.7814	0.5366	0.7978	0.6417	0.8656

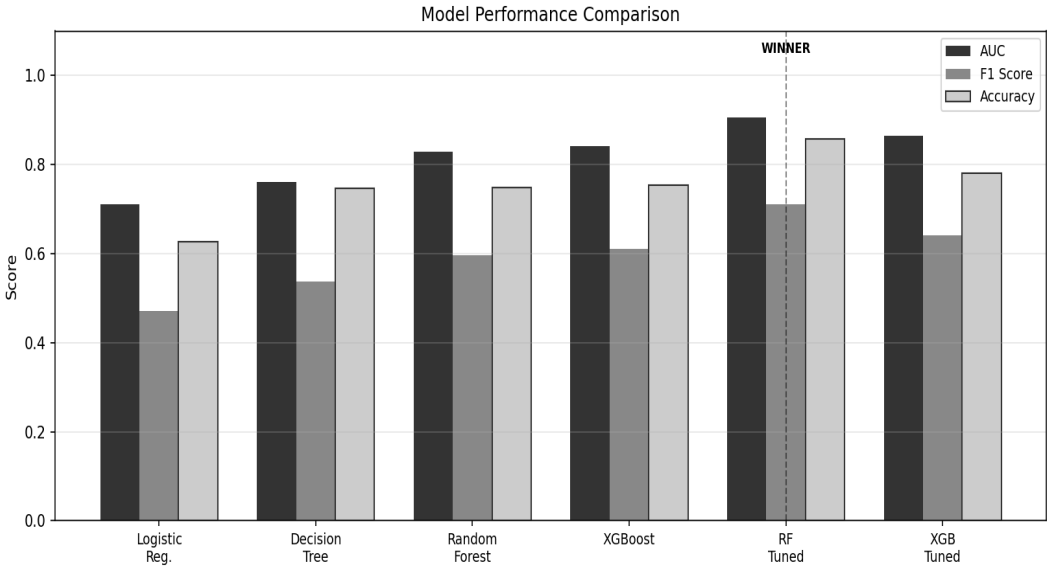

Figure 1 — Model Performance Comparison (AUC, F1, Accuracy)

3.3 Winner — RF Tuned

- Best Parameters: n_estimators=100, min_samples_leaf=2, max_features=log2, max_depth=None
- AUC = 0.9060 (Excellent — above 0.9)

- F1 Score = 0.7104, Precision = 0.7086, Recall = 0.7123
- Accuracy = 85.76%
- Confusion Matrix: TP=1374, FN=555, FP=565, TN=5369
- Selected using RandomizedSearchCV with n_iter=20, cv=5, scoring=f1

3.4 Plots

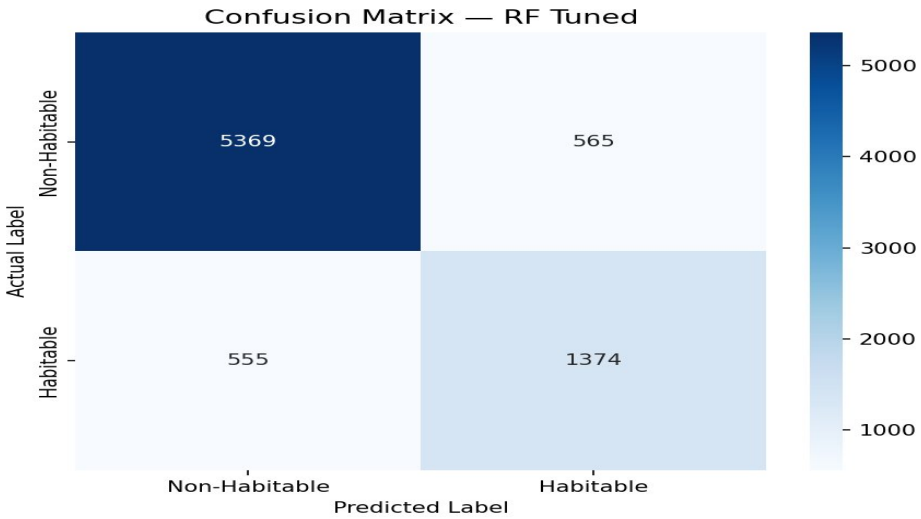

Figure 2 — Confusion Matrix (RF Tuned)

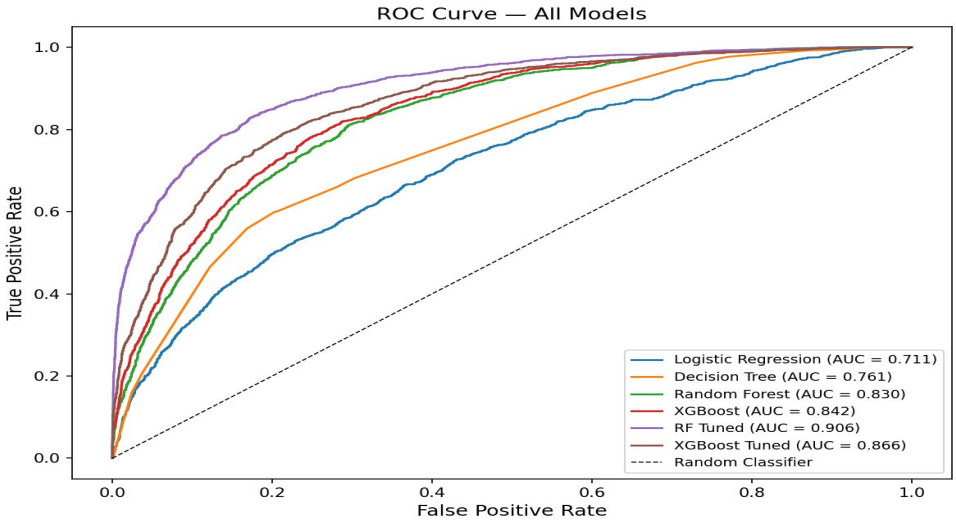

Figure 3 — ROC Curves (All Models)

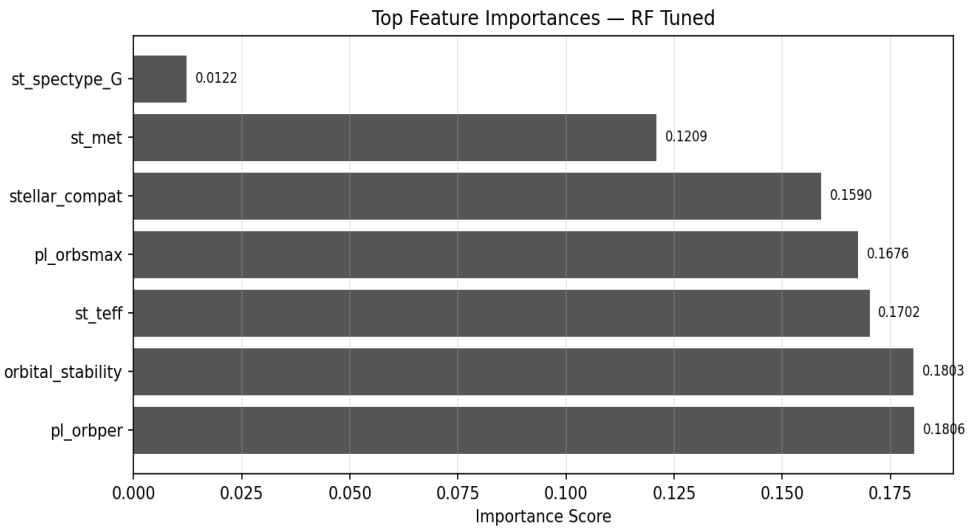

Figure 4 — Feature Importance (RF Tuned)

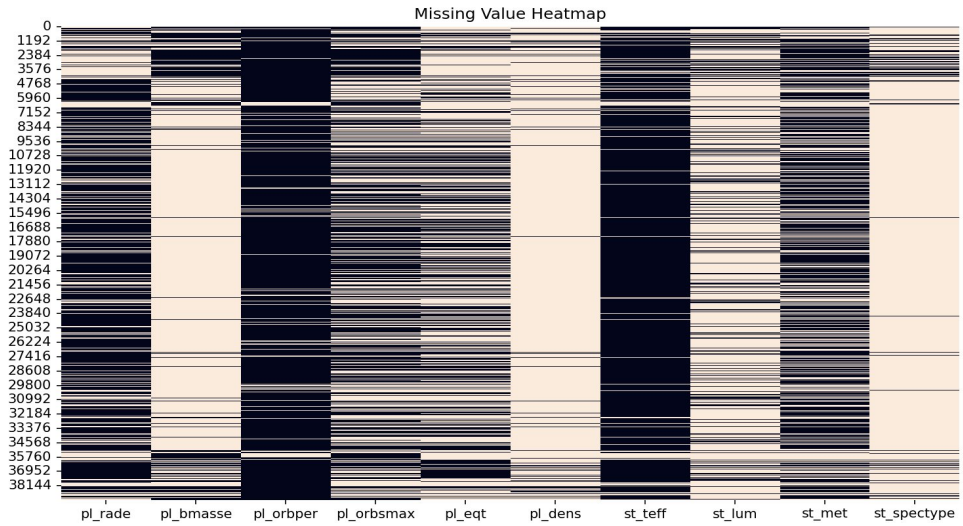

Figure 5 — Missing Value Heatmap

3.5 Feature Importance Interpretation

Feature	Importance	Scientific Meaning
pl_orbper	0.1806	Orbital period determines energy stability over time
orbital_stability	0.1803	Stable orbits allow life to develop over long periods
st_teff	0.1702	Star temperature controls UV and radiation levels
pl_orbsmax	0.1676	Orbital distance determines energy received from star
stellar_compatibility	0.1590	Sun-like stars provide stable long-lived conditions
st_met	0.1209	Higher metallicity enables rocky planet formation

End-to-End Integration Flow

4.1 Complete Data Flow

Step	Component	Action
1	User (Browser)	Opens React app at localhost:3000
2	StoryPage.jsx	Cinematic intro plays for 24 seconds
3	AuthPage.jsx	User logs in or continues as guest
4	PredictionPanel.jsx	User fills in 10 parameter fields
5	PredictionPanel.jsx	Client-side validation runs on required fields
6	Fetch API	POST /predict sent to localhost:5000
7	App.py /predict	Validates input using utils.validate_input
8	utils.build_feature_vector	Engineers features and applies scaler.pkl
9	final_model.pkl	RF Tuned model predicts habitability probability
10	App.py	Returns JSON with label, probability, confidence
11	ResultPanel.jsx	Displays verdict, 3D planet, gauge, analysis
12	localStorage	Prediction saved to history automatically

4.2 How to Run the Project

- Terminal 1: cd backend then python App.py — starts Flask on port 5000
- Terminal 2: cd frontend/exohabitai then npm start — starts React on port 3000
- Browser opens automatically at http://localhost:3000
- First visit: Story intro then login or guest then dashboard
- Subsequent visits: Auto-login from localStorage session

4.3 Sample Test — Earth-Like Planet

Parameter	Value	Meaning
Orbital Period	365.25	Earth-like yearly orbit
Semi-Major Axis	1.0	1 AU from host star
Star Temp	5778 K	Sun-like G-type star
Metallicity	0.0 dex	Solar metallicity
Spectral Type	G	Sun-like classification

Expected result: Potentially Habitable with probability above 70%. The star temperature of 5778 K falls within the 4000 to 7000 K habitable range, the G spectral type is the most favourable class, and the orbital parameters indicate a stable environment.